FuelPrint Cases guide

For Antara and Aditya · step by step, with the Codex prompts · 16 Oct 2026

What Cases does

Cases is where evidence is kept honestly. It receives the screening record from the drone capture, opens a case, records a separate inspector specimen with a seal, follows that specimen through every handover, checks the lab result with proper quality rules, writes a limited finding, and seals the whole case so anyone can check later that nothing was changed.

Three ideas run through everything you build:

1. **Different things stay different.** The drone's air reading (the screening record) and the inspector's specimen are separate items with separate IDs. One never turns into the other.
2. **Nothing is edited, only added.** A correction is a new event that points to the old one. History is never rewritten.
3. **A check proves only what it checks.** A matching hash proves the bytes did not change. It does not prove the reading was true, the factory is guilty, or the evidence is legal.

Who does what

Person	Builds	Also
Antara	The `cases` backend package: case records, intake, specimens, custody, lab maths, findings, evidence vault	Stands at laptop 3 (lab) during the demo
Aditya	All Cases screens CA01 to CA06 and the case timeline, plus the app frame	Stands at laptop 4 (inspector and vault)

The tech you will touch, in plain words

State machine. A case moves through fixed states: DRAFT, INTAKE, SPECIMEN, LAB, FINDING, SEALED. Each command is allowed only in certain states. Writing those rules down as code (and testing each one) is most of the backend work.

Append-only. Custody events are only ever added. A broken seal or a wrong entry is recorded as a new event with an exception, not by changing the old event.

SHA-256 turns any file into a 64-character fingerprint. Change one byte and the fingerprint changes completely. Java has it built in (`MessageDigest.getInstance("SHA-256")`).

Manifest. When a case is sealed, every cited record is saved as an exact byte file, and a manifest lists each file with its length and SHA-256. The manifest's own SHA-256 is stored too. Verifying means recomputing every fingerprint and comparing.

Canonical JSON. The same record must always turn into exactly the same bytes, or the fingerprints will not match. We write JSON with sorted keys and no extra spaces, and store the bytes we produced. In the finale we switch to the RFC 8785 standard and add Ed25519 signatures.

Command handlers, revisions and Flyway work the same way as in Operations. Cases migrations are numbered `V300__...` to `V399__...`.

What you get from others

From	What	Ready
Core	Server, database, `CommandBus`, `RecordStore`, blob store for exact bytes, `/ws/live`, fake replay mode, `@fp/shared`	Sat 17 Oct evening
Field	`screening_record` events (one per channel per capture, VALID, LIMITED or REJECTED with reasons)	Join day, Tue 20 Oct. Fixtures before that.
Operations	The `investigation` and `analysis` the case belongs to	Join day
Example data	`apps/shared/fixtures/*.json`	Fri 16 Oct night

What you give to others

- `case`, `specimen`, `custody_event`, `lab_result`, `finding`, `export`, `verification_report` records
- The finding that the Operations board screen needs before it can record a decision

The numbers your lab maths must match

These come from the method `TEACH-FUEL-S-1` in `contracts/reference/method-profiles.json`. Use them as test cases.

Input	Value	Result
Specimen mass	100 mg (minimum allowed 50 mg)	
Gross sulphur	2010 µg	
Blank	10 µg (limit 20 µg)	Net = 2010 − 10 = 2000 µg
Sulphur fraction	100 × 2000 / (100 × 1000)	**2.0 % w/w**
Duplicates	2.0 % and 2.2 %	RPD = 0.2 / 2.1 × 100 = **9.524 %** (limit 20 %)
Spike	100 µg added, 90 µg recovered	Recovery **90 %** (allowed 80 to 120 %)
Bad blank	30 µg	QC **fails** (over 20 µg) even if the number looks fine
Reporting limit	0.01 % w/w	Below it the result is BELOW_LOQ, never zero
Hold time	seal to analysis at most 300 simulation seconds	Over it: QC reason HOLD_TIME_EXCEEDED

If both duplicates are zero the RPD is undefined: mark it unavailable, never divide by zero. A missing QC input means PENDING, not PASS.

How to use the prompts

1. Open a terminal in the repo folder, on your own branch (`cases/antara` or `cases/aditya`).
2. Type `codex` and paste the prompt for your step.
3. Codex reads `AGENTS.md` and `contracts/AGREEMENT.md` by itself.
4. Run the check yourself. If it fails, paste the error back into Codex.
5. Read every changed file, commit, push, open a pull request.

Steps for Antara

N0 · Set up and run the fake server (Sat 17 Oct, after 6 PM)

```
git pull origin main
git checkout cases/antara && git merge main
```

```
tools/start-fake.sh
tools/test-all.sh
```

Done when the server starts and the core tests pass on your laptop.

N1 · Case records and intake (Sat 17 to Sun 18 Oct)

```
Build case records in
server/src/main/java/fuelprint/cases.
- Migration V300__cases_core.sql: tables for case,
specimen, custody_event, lab_result, finding, export,
verification_report as revision rows (AGREEMENT section
6). custody_event is insert-only.
- Command create_case (case_reviewer): takes
screening_record_ids and purpose. All ids must exist in
this run. Rejected screening records are allowed and keep
their reasons as case limitations. State INTAKE.
- A case links to the investigation of its mission.
- State machine DRAFT -> INTAKE -> SPECIMEN -> LAB ->
FINDING -> SEALED. A command in the wrong state is
REJECTED with reason WRONG_STATE. Tests: create works;
unknown screening id rejected; rejected screening record
kept with its limitation; wrong state rejected; repeated
command_id does nothing; wrong role rejected.
```

N2 · Specimens and custody (Sun 18 Oct)

```
In fuelprint.cases:
- Command create_specimen (inspector_demo): case_id,
site_id (S01 to S06), material (SYNTHETIC_COAL_COKE_LIKE),
method_id (TEACH-FUEL-S-1), container, seal_id, mass_mg.
Reject mass below 50 mg. authority_basis is always
INSPECTOR_DEMO. Creates the first custody_event: collector
holds it, seal INTACT. Case state becomes SPECIMEN.
- Command append_custody (current holder, or any
inspector_demo/lab_demo for an exception): to_holder,
seal_state (INTACT, BROKEN, MISSING), event_time_utc,
exceptions[] from: BROKEN_SEAL, MISSING_SEAL,
RECIPIENT_ABSENT, REFUSED, LATE_TRANSFER, CORRECTION. A
correction points to the event it corrects. Never update
an old event.
- A specimen id is never reused and never equals a
screening record id. Tests: chain collector -> courier ->
lab; broken seal recorded as exception and still visible;
non-holder cannot append without an exception; a
correction leaves the original row unchanged.
```

N3 · Lab result maths (Sun 18 to Mon 19 Oct)

```
In fuelprint.cases, command request_lab (lab_demo):
specimen_id, method_id, qc_inputs {gross_ug, blank_ug,
dup_a_pct, dup_b_pct, spike_added_ug, spike_recovered_ug}.
Use mass_mg from the specimen. Compute: net_ug = gross -
blank; percent_w_w = 100 * net_ug / (mass_mg*1000);
rpd_pct = |a-b| / ((a+b)/2) * 100; recovery_pct =
100*recovered/added. QC rules from TEACH-FUEL-S-1: blank
<= 20 ug, RPD <= 20 %, recovery 80-120 %, hold time seal-
>analysis <= 300 sim seconds, method must match material.
Below 0.01 % w/w -> reporting BELOW_LOQ. Missing input ->
PENDING. Mean zero for RPD -> unavailable, no division.
Store raw inputs, computed values and every failed reason.
Case state LAB. Write the exact tests from the table in
docs/plan/03-cases-guide.md: 2.0 %,
9.5238 % (4 decimals), 90 %, blank 30 fails, below-LOQ,
zero duplicates, missing spike = PENDING, hold time
exceeded.
```

N4 · Findings (Mon 19 Oct)

```
In fuelprint.cases, command revise_finding
(case_reviewer): outcome, cited_refs[], alternatives[],
limitations[].
- SUPPORTED_WITHIN_SCOPE needs at least one cited VALID
screening record or a lab_result with qc_status PASS.
- REJECTED_FOR_USE needs a cited REJECTED item.
- INCONCLUSIVE is always allowed.
- limitations must not be empty. If the cited analysis
outcome is not PROVISIONAL_SINGLE, alternatives must not
be empty.
- legal_suitability is always NOT_ASSESSED; reject any
attempt to set it.
- Reject finding text containing "guilty", "illegal",
"proved" or "coal used".
- A new finding is a new revision; old ones stay. Case
state FINDING. Tests for each rule.
```

N5 · Evidence vault (Mon 19 Oct)

```
In fuelprint.cases:
- Command seal_export (exporter): case must be in FINDING.
Collect every record the finding cites, plus the case,
specimens, custody events and lab results. Serialise each
as canonical JSON (sorted keys, no spaces, UTF-8), store
the exact bytes in the core blob store at
payload/records/<type>/<id>-r<rev>.json. Build the
manifest {schema:"fp.evidence/1", case_id, case_revision,
created_utc, members:[{path,length,sha256}], limitations}.
Store its exact bytes and manifest_sha256. Case state
SEALED. Never re-serialise later.
- Command verify_export: recompute every member hash and
the manifest hash from stored bytes. Save a
verification_report with seven axes: integrity
(PASS/FAIL), key_trust (NOT_SIGNED until the finale),
lineage (PASS if every cited ref is a member),
trusted_time (UNVERIFIED), custody_documentation (PASS, or
LIMITED if any custody exception),
measurement_qualification (SYNTHETIC_SCOPE_ONLY),
legal_suitability (NOT_ASSESSED). List reasons and which
member failed.
- Command tamper_copy: copy the export into a NEW export
marked TAMPER_DEMO_COPY, flip one byte of one member's
copied bytes. The original export and its blobs must stay
byte-identical. Tests: seal then verify passes; tamper
copy fails integrity on exactly that member; original
still passes after tampering the copy.
```

N6 · Join day (Tue 20 Oct)

Run SC1, SC2 and SC3 end to end with the real server and Field's real screening records. Check that the SC2 case ends REJECTED_FOR_USE and the SC3 case ends INCONCLUSIVE.

Steps for Aditya

D0 · Set up (Sat 17 Oct evening)

```
git pull origin main
git checkout cases/aditya && git merge main
tools/start-fake.sh
tools/dev-app.sh cases       #
http://localhost:5173/cases/
```

D1 · App frame and case list (Sat 17 Oct)

```
Build the Cases app frame in apps/cases with @fp/shared
(live connection, station login, StaleBanner, Explain).
- Top bar: run, case id short code, case state, connection
status, roles.
- Case list page /cases/: every case with state, outcome
so far and next responsible role.
- Routes for CA01 to CA06 as placeholders.
- A case timeline component used on every case page: every
record and custody event in time order with who did it.
Corrections show next to what they correct. Vitest tests
for the timeline order and for the stale state.
```

D2 · Intake screen CA01 (Sun 18 Oct)

```
CA01 at /cases/:id/intake (case_reviewer):
- Screening records per channel with decision VALID /
LIMITED / REJECTED and reasons, volume and valid seconds.
Rejected ones are shown, never hidden.
- Link to the investigation and its latest analysis
outcome.
- "Create case" from selected screening records
(create_case).
- A clear line: "This is an air reading from the drone. It
is not a specimen."
```

D3 · Specimen and custody screens CA02, CA03 (Sun 18 Oct)

```
CA02 /cases/:id/specimens/new (inspector_demo): form for
site (S01–S06), material, method, container, seal id, mass
in mg. Show the server's rejection reason (for example
mass below 50 mg) next to the field. CA03
/cases/specimens/:id/custody: the chain as a vertical list
of holders with seal state icons and text (not colour
alone). "Hand over" form for the current holder. "Record
exception" form with the exception types. Corrections
appear beside the original, which stays visible.
```

D4 · Lab screen CA04 (Mon 19 Oct)

```
CA04 /cases/specimens/:id/lab (lab_demo):
– Form for gross, blank, duplicates, spike added and
recovered.
– Two "fill example" buttons: "valid example" (2010, 10,
2.0, 2.2, 100, 90) and "blank fails" (blank 30).
– After the server answers: raw values, computed values,
each QC rule with pass/fail and the limit, reporting
status, hold time used.
– Line: "Synthetic teaching method. A valid result
describes this specimen only. It does not show that this
material caused the plume."
```

D5 · Finding screen CA05 (Mon 19 Oct)

```
CA05 /cases/:id/finding (case_reviewer): choose outcome,
tick cited records, write alternatives and limitations.
Show the server's rule errors in plain words. Legal
suitability is shown as "Not assessed" and cannot be
changed.
```

D6 · Vault screen CA06 (Mon 19 Oct)

```
CA06 /cases/:id/vault (exporter):
– Seal button, then the member list with path, size and
the first 12 characters of each SHA-256.
– Verify button: show the seven axes as seven separate
rows with their own status and one-line meaning. Never one
big green "verified" badge.
– Tamper demo button: makes a copy, changes one byte,
verifies the copy. Show the failed member highlighted and
that the original still verifies.
```

D7 · Join day (Tue 20 Oct)

Walk all three scenarios on laptops 3 and 4 and note every mismatch with a screenshot.

Finale work (25 to 29 Oct)

Item	Who
Ed25519 signing of the exact manifest bytes, key kept outside the repo, public key fingerprint shown on screen	Antara
Export as a ZIP with size limits and safe paths, plus a Java command-line verifier run on another laptop	Antara
Aerosol filter path (TEACH-AEROSOL-S-1): separate media item, loading below LOQ shown as limited	Antara
Case replay: step through a closed case's timeline	Aditya
Signature and key-trust rows on CA06, and the second-laptop verify demo	Aditya
Polish, keyboard-only pass, 200 % zoom check	Aditya

Done means

- Your tests pass and your pull request is merged.
- The three scenarios end supported, rejected and inconclusive on your screens.
- Tampering a copy fails, and the original still passes.
- You can explain to a judge, in two sentences, what a passing verify proves and what it does not.